

RTL:

1. Top Module:

```
1  `include "uvm/includes/all_imports.svh"
2
3  module top();
4
5      bit clk;
6
7      initial begin
8          clk = 0;
9          forever
10             #1 clk = ~clk;
11      end
12
13      alsu_if alsuif (clk);
14
15      ALSU DUT (.clk(clk), .A(alsuif.A), .B(alsuif.B),
16              .cin(alsuif.cin), .serial_in(alsuif.serial_in),
17              .red_op_A(alsuif.red_op_A), .red_op_B(alsuif.red_op_B),
18              .opcode(alsuif.opcode), .bypass_A(alsuif.bypass_A),
19              .bypass_B(alsuif.bypass_B), .rst(alsuif.rst), .direction(alsuif.direction),
20              .leds(alsuif.leds), .out(alsuif.out));
21
22      //bind the SVA module to the designs
23      bind ALSU ALSU_SVA ALSU_SVA_inst(.clk(clk), .A(alsuif.A), .B(alsuif.B),
24              .cin(alsuif.cin), .serial_in(alsuif.serial_in),
25              .red_op_A(alsuif.red_op_A), .red_op_B(alsuif.red_op_B),
26              .opcode(alsuif.opcode), .bypass_A(alsuif.bypass_A),
27              .bypass_B(alsuif.bypass_B), .rst(alsuif.rst), .direction(alsuif.direction),
28              .leds(alsuif.leds), .out(alsuif.out));
29
30
31      initial begin
32          uvm_config_db#(virtual alsu_if)::set(null, "uvm_test_top", "vif", alsuif);
33          run_test("alsu_test");
34      end
35
36  endmodule
```

2. Interface

```
1 interface alsu_if(clk);
2     input clk;
3     logic cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
4     logic [2:0] opcode;
5     logic signed [2:0] A, B;
6     logic [15:0] leds;
7     logic signed [5:0] out;
8 endinterface : alsu_if
```

3. Assertions

```
1 import alsu_seq_item_pkg::*;
2 module ALSU_SVA(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
3
4 input logic clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5 input logic [2:0] opcode;
6 input logic signed [2:0] A, B;
7 input logic [15:0] leds;
8 input logic signed [5:0] out;
9
10 logic invalid_opcode, invalid_red_op, invalid;
11
12 assign invalid_red_op = (red_op_A | red_op_B) & (opcode[1] | opcode[2]);
13 assign invalid_opcode = opcode[1] & opcode[2];
14 assign invalid = invalid_red_op | invalid_opcode;
15
16 // Reset behavior
17 always_comb begin
18     //5. When the asynchronous reset is asserted then the counter output is tied to low at the same instant.
19     if(rst) begin
20         assert final (out == 0);
21         cover final (leds == 0);
22     end
23 end
24
25 // Invalid case: out must be zero
26 property p_invalid_out_zero;
27     @(posedge clk) disable iff (rst) (invalid && !bypass_A && !bypass_B) |-> ##2 (out == 0);
28 endproperty
29
30 property p_valid_leds_zero;
31     @(posedge clk) disable iff (rst) (!invalid) |> (leds == 0);
32 endproperty
33
34 property bypassA_prop;
35     @(posedge clk) disable iff (rst) bypass_A |>##1 (out == $past(A,2));
36 endproperty
37
38 property bypassB_prop;
39     @(posedge clk) disable iff (rst) bypass_B && !bypass_A |>##1 (out == $past(B,2));
40 endproperty
41
42
43
44
```

```

1
2
3     sequence NO_bypass;
4     !(bypass_A || bypass_B);
5     endsequence
6
7     // Red_op_A assertion
8     property red_op_A_xor;
9     @(posedge clk) disable iff(rst)
10        NO_bypass |->(red_op_A && opcode==XOR) |> ##1 (out == $past(^A,2));
11    endproperty
12
13    property red_op_A_or;
14    @(posedge clk) disable iff(rst)
15        NO_bypass |->(red_op_A && opcode==OR) |> ##1 (out == $past(|A,2));
16    endproperty
17
18
19    // Red_op_B assertion - XOR
20    property red_op_B_xor;
21    @(posedge clk) disable iff(rst)
22        NO_bypass |-> (!red_op_A && red_op_B && opcode==XOR) |> ##1 (out == $past(^B,2));
23    endproperty
24
25    // Red_op_B assertion - OR
26    property red_op_B_or;
27    @(posedge clk) disable iff(rst)
28        NO_bypass |-> (!red_op_A && red_op_B && opcode==OR) |> ##1 (out == $past(|B,2));
29    endproperty
30
31
32    sequence NO_bypassANDred;
33    !(bypass_A || bypass_B || red_op_A || red_op_B );
34    endsequence
35
36    property OR_prop;
37    @(posedge clk) disable iff(rst)
38        NO_bypassANDred |-> (opcode==OR) |> ##1 (out == ($past(A,2) | $past(B,2)));
39    endproperty
40
41    // Normal XOR (no reduction)
42    property XOR_prop;
43    @(posedge clk) disable iff(rst)
44        NO_bypassANDred |-> (opcode==XOR) |> ##1 (out == ($past(A,2) ^ $past(B,2)));
45    endproperty
46
47    // ADD
48    property ADD_prop;
49    @(posedge clk) disable iff(rst)
50        NO_bypassANDred |-> (opcode==ADD) |> ##1
51        out == ($past(cin,2)? $past(A,2) + $past(B,2)+3'd1 :$past(A,2) + $past(B,2));
52    endproperty
53
54    // MUL
55    property MUL_prop;
56    @(posedge clk) disable iff(rst)
57        NO_bypassANDred |-> (opcode==MULT ) |> ##1
58        (out == ($past(A,2) * $past(B,2)));
59    endproperty
60

```

```

1
2 // SHIFT (opcode = 100)
3 property SHIFT_R_prop;
4 @(posedge clk) disable iff(rst)
5     NO_bypassANDred |-> (opcode==SHIFT && direction) |> ##1
6     (out == { $past(out[4:0],1), $past(serial_in,2) });
7 endproperty
8
9 // ROTATE (opcode = 101)
10 property ROTATE_R_prop;
11 @(posedge clk) disable iff(rst)
12     NO_bypassANDred |-> (opcode==ROTATE && direction) |> ##1
13     (out == { $past(out[4:0],1), $past(out[5],1) });
14 endproperty
15
16 // SHIFT (opcode = 100)
17 property SHIFT_L_prop;
18 @(posedge clk) disable iff(rst)
19     NO_bypassANDred |-> (opcode==SHIFT && !direction) |> ##1
20     (out == { $past(serial_in,2), $past(out[5:1],1) });
21 endproperty
22
23
24 // ROTATE (opcode = 101)
25 property ROTATE_L_prop;
26 @(posedge clk) disable iff(rst)
27     NO_bypassANDred |-> (opcode==ROTATE && !direction) |> ##1
28     (out == { $past(out[0],1), $past(out[5:1],1) });
29 endproperty
30
31
32 property invalid_prop;
33     @(posedge clk) disable iff(rst)
34     (opcode== INVALID_6 || opcode==INVALID_7)
35     |> ((red_op_A||red_op_B)&& ~(opcode== OR ||opcode== XOR ))
36     |> ##1(leds == ~$past(leds));
37 endproperty
38
39 //-----
40 // Assertions
41 //-----
42
43 p_invalid_out_zero_assertion          : assert property (p_invalid_out_zero);
44 p_valid_leds_zero_assertion           : assert property (p_valid_leds_zero);
45 bypassA_assertion                    : assert property( bypassA_prop);
46 bypassB_assertion                    : assert property( bypassB_prop);
47 red_op_A_xor_assertion                : assert property(red_op_A_xor);
48 red_op_A_or_assertion                 : assert property(red_op_A_or);
49 red_op_B_xor_assertion                : assert property(red_op_B_xor);
50 red_op_B_or_assertion                 : assert property(red_op_B_or);
51 OR_assertion                         : assert property(OR_prop);
52 XOR_assertion                        : assert property(XOR_prop);
53 ADD_assertion                        : assert property(ADD_prop);
54 MUL_assertion                        : assert property(MUL_prop);
55 SHIFT_R_assertion                    : assert property(SHIFT_R_prop);
56 ROTATE_R_assertion                   : assert property(ROTATE_R_prop);
57 SHIFT_L_assertion                    : assert property(SHIFT_L_prop);
58 ROTATE_L_assertion                   : assert property(ROTATE_L_prop);
59 invalid_assertion                    : assert property(invalid_prop);
60
61
62
63 //-----
64 // Coverage
65 //-----
66
67 p_invalid_out_zero_coverage           : cover property (p_invalid_out_zero);
68 p_valid_leds_zero_coverage           : cover property (p_valid_leds_zero);
69 bypassA_coverage                     : cover property( bypassA_prop);
70 bypassB_coverage                     : cover property( bypassB_prop);
71 red_op_A_xor_coverage                : cover property(red_op_A_xor);
72 red_op_A_or_coverage                 : cover property(red_op_A_or);
73 red_op_B_xor_coverage                : cover property(red_op_B_xor);
74 red_op_B_or_coverage                 : cover property(red_op_B_or);
75 OR_coverage                         : cover property(OR_prop);
76 XOR_coverage                        : cover property(XOR_prop);
77 ADD_coverage                        : cover property(ADD_prop);
78 MUL_coverage                        : cover property(MUL_prop);
79 SHIFT_R_coverage                     : cover property(SHIFT_R_prop);
80 ROTATE_R_coverage                    : cover property(ROTATE_R_prop);
81 SHIFT_L_coverage                     : cover property(SHIFT_L_prop);
82 ROTATE_L_coverage                    : cover property(ROTATE_L_prop);
83 invalid_coverage                     : cover property(invalid_prop);
84
85
86 endmodule

```

UVM files

1. Test

```
1  package alsu_test_pkg;
2
3      import uvm_pkg::*;
4      `include "uvm_macros.svh"
5
6      import alsu_env_pkg::*;
7      import alsu_config_pkg::*;
8      import alsu_seq_pkg::*;
9
10
11     class alsu_test extends uvm_test;
12         `uvm_component_utils(alsu_test)
13
14         alsu_env env;
15         alsu_config alsu_config_obj_test;
16         alsu_reset_seq reset_seq;
17         alsu_main_seq main_seq;
18         alsu_directed_seq direct_seq;
19
20         function new(string name = "alsu_test", uvm_component parent = null);
21             super.new(name, parent);
22         endfunction
23
24         function void build_phase(uvm_phase phase);
25             super.build_phase(phase);
26             env = alsu_env::type_id::create("env", this);
27             reset_seq = alsu_reset_seq::type_id::create("reset_seq");
28             main_seq = alsu_main_seq::type_id::create("main_seq");
29             direct_seq = alsu_directed_seq::type_id::create("direct_seq");
30             alsu_config_obj_test = alsu_config::type_id::create("alsu_config_obj_test");
31             if(!uvm_config_db #(virtual alsu_if)::get(this, "", "vif", alsu_config_obj_test.alsu_config_vif))
32                 `uvm_fatal("build_phase", "Test - Unable to get Alsu Virtual Interface");
33
34             uvm_config_db#(alsu_config)::set(null, "*", "CFG", alsu_config_obj_test);
35         endfunction
36
37         task run_phase(uvm_phase phase);
38             super.run_phase(phase);
39             phase.raise_objection(this);
40
41             `uvm_info("run_phase", "Reset Asserted", UVM_MEDIUM);
42             reset_seq.start(env.agt.sqr);
43             `uvm_info("run_phase", "Reset Deasserted", UVM_MEDIUM);
44
45             `uvm_info("run_phase", "Main Sequencer Begin", UVM_MEDIUM);
46             main_seq.start(env.agt.sqr);
47             `uvm_info("run_phase", "Main Sequencer end", UVM_MEDIUM);
48
49             `uvm_info("run_phase", "Directed Sequencer Begin", UVM_MEDIUM);
50             direct_seq.start(env.agt.sqr);
51             `uvm_info("run_phase", "Directed Sequencer end", UVM_MEDIUM);
52
53             phase.drop_objection(this);
54         endtask
55     endclass
56
57 endpackage
```

2. Environment



```
1  package alsu_env_pkg;
2
3      import uvm_pkg::*;
4      `include "uvm_macros.svh"
5
6      import alsu_agent_pkg::*;
7      import alsu_scoreboard_pkg::*;
8      import alsu_CovCollector_pkg::*;
9
10     class alsu_env extends uvm_env;
11         `uvm_component_utils(alsu_env);
12
13         alsu_agent agt;
14         alsu_scoreboard sb;
15         alsu_covcollector cov;
16
17         function new(string name = "alsu_env", uvm_component parent = null);
18             super.new(name, parent);
19         endfunction
20
21         function void build_phase(uvm_phase phase);
22             super.build_phase(phase);
23             agt = alsu_agent::type_id::create("agt", this);
24             sb = alsu_scoreboard::type_id::create("sb", this);
25             cov = alsu_covcollector::type_id::create("cov", this);
26         endfunction
27
28         function void connect_phase(uvm_phase phase);
29             super.connect_phase(phase);
30             agt.agt_ap.connect(sb.sb_export);
31             agt.agt_ap.connect(cov.cov_export);
32         endfunction
33
34     endclass
35
36 endpackage
```

3. Configuration

```
1 package alsu_config_pkg;
2     import uvm_pkg::*;
3     `include "uvm_macros.svh"
4
5     class alsu_config extends uvm_object;
6         `uvm_object_utils(alsu_config)
7         // Configuration parameters
8         virtual alsu_if alsu_config_vif;
9
10        function new(string name = "alsu_config");
11            super.new(name);
12        endfunction
13    endclass : alsu_config
14 endpackage
```

4. Sequence item

```
1 package alsu_seq_item_pkg;
2
3     import uvm_pkg::*;
4     `include "uvm_macros.svh"
5
6     typedef enum logic [2:0] {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7} OP_CODE_e;
7
8     parameter INPUT_PRIORITY = "A";
9     parameter FULL_ADDER = "ON";
10    parameter MAXPOS = 3'b011;
11    parameter MAXNEG = 3'b100;
12    parameter ZERO = 3'b000;
13
14    class alsu_seq_item extends uvm_sequence_item;
15        `uvm_object_utils(alsu_seq_item)
16
17        rand logic cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
18        rand OP_CODE_e opcode;
19        rand logic signed [2:0] A, B;
20        rand logic [15:0] leds;
21        logic signed [5:0] out;
22
23        constraint c_rst{
24            rst dist{0:/95, 1:/5};
25        }
26
27        constraint c_inputs{
28            if(opcode != SHIFT || opcode != ROTATE){
29                //ALSU_2
30                A dist{MAXNEG:/30, ZERO:/30, MAXPOS:/30, [-3:-1]/10, [1:2]/10};
31                B dist{MAXNEG:/30, ZERO:/30, MAXPOS:/30, [-3:-1]/10, [1:2]/10};
32
33                if(((opcode == OR) || (opcode == XOR) && red_op_A)){
34                    |A == 1;
35                    B == 3'b000;
36                }
37
38                if(((opcode == OR) || (opcode == XOR) && red_op_B)){
39                    |B == 1;
40                    A == 3'b000;
41                }
42            }
43        }
44
45        constraint c_opcode{
46            opcode dist{[OR:ROTATE]/90, [INVALID_6: INVALID_7]/10};
47        }
48
49        constraint c_bypass{
50            bypass_A dist{0:/90, 1:/10};
51            bypass_B dist{0:/90, 1:/10};
52        }
53
54        function new(string name = "alsu_seq_item");
55            super.new(name);
56        endfunction //new()
57
58        function string convert2string();
59            return $formatf("%s | rst = 0b%b, cin = 0b%b, red_op_A = 0b%b, red_op_B = 0b%b, bypass_A = 0b%b, bypass_B = 0b%b, direction = 0b%b, serial_in = 0b%b, opcode = %s, A = %s, B = %s, leds = 0b%b",
60                super.convert2string(), rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in, opcode, A, B, leds, out);
61        endfunction
62
63        function string convert2string_stimulus();
64            return $formatf(" rst = 0b%b, cin = 0b%b, red_op_A = 0b%b, red_op_B = 0b%b, bypass_A = 0b%b, bypass_B = 0b%b, direction = 0b%b, serial_in = 0b%b, opcode = %s, A = %s, B = %s",
65                rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in, opcode, A, B);
66        endfunction
67
68    endclass //alsu_seq_item extends uvm_sequence_item
69 endpackage
```

5. Sequences

Main and Reset Sequence

```
1 package aslu_seq_pkg;
2
3 import uvm_pkg::*;
4 import alsu_seq_item_pkg::*;
5 `include "uvm_macros.svh"
6
7 class alsu_reset_seq extends uvm_sequence #(alsu_seq_item);
8   `uvm_object_utils(alsu_reset_seq)
9
10    alsu_seq_item seq_item;
11    function new(string name = "alsu_reset_seq");
12        super.new(name);
13    endfunction //new()
14
15    virtual task body();
16        seq_item = alsu_seq_item::type_id::create("seq_item");
17        start_item(seq_item);
18        seq_item.rst = 1;
19        seq_item.opcode = OR;
20        seq_item.A = 0;
21        seq_item.B = 0;
22        seq_item.bypass_A = 0;
23        seq_item.bypass_B = 0;
24        seq_item.red_op_A = 0;
25        seq_item.red_op_B = 0;
26        seq_item.direction = 0;
27        seq_item.serial_in = 0;
28        seq_item.cin = 0;
29        finish_item(seq_item);
30    endtask
31 endclass //alsu_reset_seq extends uvm_sequence
32
33 class alsu_main_seq extends uvm_sequence #(alsu_seq_item);
34   `uvm_object_utils(alsu_main_seq)
35
36    alsu_seq_item seq_item;
37
38    function new(string name = "alsu_main_seq");
39        super.new(name);
40    endfunction //new()
41
42    virtual task body();
```


Directed Sequence

```
1
2   class alsu_directed_seq extends uvm_sequence #(alsu_seq_item);
3       `uvm_object_utils(alsu_directed_seq)
4
5       alsu_seq_item seq_item;
6
7       function new(string name = "alsu_directed_seq");
8           super.new(name);
9       endfunction //new()
10
11       virtual task body();
12           OPCODE_e opcode = OR;
13
14           //Bins Transition in ALU_CP
15           for(int i = 0; i <= 6; i++)begin
16               seq_item = alsu_seq_item::type_id::create("seq_item");
17               start_item(seq_item);
18               assert(seq_item.randomize());
19               seq_item.rst          = 0;
20               seq_item.rst          = 0;
21               seq_item.red_op_A     = 0;
22               seq_item.red_op_B     = 0;
23               seq_item.opcode       = opcode;
24               finish_item(seq_item);
25               opcode               = opcode.next;
26           end
27
28           //Cross coverage REP_OP_B_OR
29           seq_item = alsu_seq_item::type_id::create("seq_item");
30           start_item(seq_item);
31           seq_item.rst = 0;
32           seq_item.opcode = XOR;
33           seq_item.A = 0;
34           seq_item.B = 3'b001;
35           seq_item.bypass_A = 0;
36           seq_item.bypass_B = 0;
37           seq_item.red_op_A = 0;
38           seq_item.red_op_B = 1;
39           seq_item.direction = 0;
40           seq_item.serial_in = 0;
41           seq_item.cin = 0;
42           finish_item(seq_item);
43
44           seq_item = alsu_seq_item::type_id::create("seq_item");
45           start_item(seq_item);
46           seq_item.rst = 0;
47           seq_item.opcode = OR;
48           seq_item.A = 0;
49           seq_item.B = 3'b010;
50           seq_item.bypass_A = 0;
51           seq_item.bypass_B = 0;
52           seq_item.red_op_A = 0;
53           seq_item.red_op_B = 1;
54           seq_item.direction = 0;
55           seq_item.serial_in = 0;
56           seq_item.cin = 0;
57           finish_item(seq_item);
58
```

```

58
59         //Assertion coverage
60         seq_item = alsu_seq_item::type_id::create("seq_item");
61         start_item(seq_item);
62         seq_item.rst = 0;
63         seq_item.opcode = OR;
64         seq_item.A = 3'b011;
65         seq_item.B = 3'b010;
66         seq_item.bypass_A = 0;
67         seq_item.bypass_B = 0;
68         seq_item.red_op_A = 1;
69         seq_item.red_op_B = 0;
70         seq_item.direction = 0;
71         seq_item.serial_in = 0;
72         seq_item.cin = 0;
73         finish_item(seq_item);
74
75         seq_item = alsu_seq_item::type_id::create("seq_item");
76         start_item(seq_item);
77         seq_item.rst = 0;
78         seq_item.opcode = OR;
79         seq_item.A = 3'b001;
80         seq_item.B = 3'b010;
81         seq_item.bypass_A = 0;
82         seq_item.bypass_B = 0;
83         seq_item.red_op_A = 0;
84         seq_item.red_op_B = 1;
85         seq_item.direction = 0;
86         seq_item.serial_in = 0;
87         seq_item.cin = 0;
88         finish_item(seq_item);
89
90
91     endtask
92 endclass //alsu_directed_seq extends uvm_sequence
93
94 endpackage

```

6. Sequencer

```
1 package alsu_sequencer_pkg;
2
3     import uvm_pkg::*;
4     `include "uvm_macros.svh"
5     import alsu_seq_item_pkg::*;
6
7     class alsu_sequencer extends uvm_sequencer #(alsu_seq_item);
8         `uvm_component_utils(alsu_sequencer)
9         function new(string name = "alsu_sequencer", uvm_component parent = null);
10             super.new(name, parent);
11         endfunction //new()
12     endclass //alsu_sequencer extends uvm_sequencer #(alsu_seq_item)
13 endpackage
```

7. Agent

```
1 package alsu_agent_pkg;
2
3     import uvm_pkg::*;
4     `include "uvm_macros.svh"
5     import alsu_driver_pkg::*;
6     import alsu_sequencer_pkg::*;
7     import alsu_monitor_pkg::*;
8     import alsu_config_pkg::*;
9     import alsu_seq_item_pkg::*;
10
11
12     class alsu_agent extends uvm_agent;
13         `uvm_component_utils(alsu_agent)
14
15         alsu_config alsu_cfg;
16         alsu_driver drv;
17         alsu_monitor mon;
18         alsu_sequencer sqr;
19         uvm_analysis_port #(alsu_seq_item) agt_ap;
20
21
22         function new(string name = "alsu_agent", uvm_component parent = null);
23             super.new(name, parent);
24         endfunction //new()
25     endclass
```



```
1
2  function void build_phase(uvm_phase phase);
3      super.build_phase(phase);
4      drv = alsu_driver::type_id::create("drv", this);
5      mon = alsu_monitor::type_id::create("mon", this);
6      sqr = alsu_sequencer::type_id::create("sqr", this);
7      agt_ap = new("agt_ap", this);
8      if(!uvm_config_db #(alsu_config)::get(this, "", "CFG", alsu_cfg))begin
9          `uvm_fatal("build_phase", "Error, Unable to get configuration object")
10     end
11 endfunction
12
13 function void connect_phase(uvm_phase phase);
14     super.connect_phase(phase);
15     //driver connections
16     drv.alsu_vif = alsu_cfg.alsu_config_vif;
17     drv.seq_item_port.connect(sqr.seq_item_export);
18
19     //monitor connections
20     mon.alsu_vif = alsu_cfg.alsu_config_vif;
21     mon.mon_ap.connect(agt_ap);
22 endfunction
23 endclass //alsu_agent extends uvm_agent
24
25 endpackage
```

8. Driver

```
1 package alsu_driver_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5 import alsu_seq_item_pkg::*;
6
7
8 class alsu_driver extends uvm_driver #(alsu_seq_item);
9     `uvm_component_utils(alsu_driver);
10
11     virtual alsu_if alsu_vif;
12     alsu_seq_item stim_seq_item;
13
14
15     function new(string name = "alsu_driver", uvm_component parent);
16         super.new(name, parent);
17     endfunction
18
19
20     task run_phase(uvm_phase phase);
21         super.run_phase(phase);
22         forever begin
23             stim_seq_item = alsu_seq_item::type_id::create("stim_seq_item");
24             seq_item_port.get_next_item(stim_seq_item);
25             alsu_vif.rst = stim_seq_item.rst;
26             alsu_vif.cin = stim_seq_item.cin;
27             alsu_vif.red_op_A = stim_seq_item.red_op_A;
28             alsu_vif.red_op_B = stim_seq_item.red_op_B;
29             alsu_vif.bypass_A = stim_seq_item.bypass_A;
30             alsu_vif.bypass_B = stim_seq_item.bypass_B;
31             alsu_vif.direction = stim_seq_item.direction;
32             alsu_vif.serial_in = stim_seq_item.serial_in;
33             alsu_vif.opcode = stim_seq_item.opcode;
34             alsu_vif.A = stim_seq_item.A;
35             alsu_vif.B = stim_seq_item.B;
36             repeat(2)@(negedge alsu_vif.clk);
37             seq_item_port.item_done();
38             `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
39         end
40     endtask
41 endclass
42
43 endpackage
```

9. Monitor

```
1  package alsu_monitor_pkg;
2
3      import uvm_pkg::*;
4      `include "uvm_macros.svh"
5      import alsu_seq_item_pkg::*;
6
7
8      class alsu_monitor extends uvm_monitor;
9          `uvm_component_utils(alsu_monitor);
10
11          virtual alsu_if alsu_vif;
12          alsu_seq_item rsp_seq_item;
13          uvm_analysis_port #(alsu_seq_item) mon_ap;
14
15
16          function new(string name = "alsu_monitor", uvm_component parent);
17              super.new(name, parent);
18          endfunction
19
20          function void build_phase(uvm_phase phase);
21              super.build_phase(phase);
22              mon_ap = new("mon_ap", this);
23          endfunction
24
25
26          task run_phase(uvm_phase phase);
27              super.run_phase(phase);
28              forever begin
29                  rsp_seq_item = alsu_seq_item::type_id::create("rsp_seq_item");
30                  repeat(2)@(negedge alsu_vif.clk);
31                  rsp_seq_item.rst = alsu_vif.rst;
32                  rsp_seq_item.cin = alsu_vif.cin;
33                  rsp_seq_item.red_op_A = alsu_vif.red_op_A;
34                  rsp_seq_item.red_op_B = alsu_vif.red_op_B;
35                  rsp_seq_item.bypass_A = alsu_vif.bypass_A;
36                  rsp_seq_item.bypass_B = alsu_vif.bypass_B;
37                  rsp_seq_item.direction = alsu_vif.direction;
38                  rsp_seq_item.serial_in = alsu_vif.serial_in;
39                  rsp_seq_item.opcode = OPPOSITE_e'(alsu_vif.opcode);
40                  rsp_seq_item.A = alsu_vif.A;
41                  rsp_seq_item.B = alsu_vif.B;
42                  rsp_seq_item.leds = alsu_vif.leds;
43                  rsp_seq_item.out = alsu_vif.out;
44                  mon_ap.write(rsp_seq_item);
45                  `uvm_info("run_phase", rsp_seq_item.convert2string_stimulus(), UVM_HIGH)
46              end
47          endtask
48      endclass
49
50  endpackage
```

10. Scoreboard:

```
1 package alsu_scoreboard_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5
6 import alsu_seq_item_pkg::*;
7 parameter INPUT_PRIORITY = "A";
8 parameter FULL_ADDER = "ON";
9
10 class alsu_scoreboard extends uvm_scoreboard;
11     `uvm_component_utils(alsu_scoreboard)
12
13     uvm_analysis_export #(alsu_seq_item) sb_export;
14     uvm_tlm_analysis_fifo #(alsu_seq_item) sb_fifo;
15     alsu_seq_item seq_item;
16     // logic [15:0] leds_ref = 0;
17     logic signed [5:0] out_ref = 0;
18
19     int error_count = 0;
20     int correct_count = 0;
21
22     function new(string name = "alsu_scoreboard", uvm_component parent = null);
23         super.new(name, parent);
24     endfunction //new()
25
26     function void build_phase(uvm_phase phase);
27         super.build_phase(phase);
28         sb_export = new("sb_export", this);
29         sb_fifo = new("sb_fifo", this);
30     endfunction
31
32     function void connect_phase(uvm_phase phase);
33         super.connect_phase(phase);
34         sb_export.connect(sb_fifo.analysis_export);
35     endfunction
36
37     task run_phase(uvm_phase phase);
38         super.run_phase(phase);
39         forever begin
40             sb_fifo.get(seq_item);
41             ref_model(seq_item);
42             if(seq_item.out != out_ref)begin
43                 `uvm_error("run_phase", "Mismatch is scoreboard output");
44                 error_count++;
45             end
46             else begin
47                 correct_count++;
48             end
49         end
50     endtask
51
52     function void report_phase(uvm_phase phase);
53         super.report_phase(phase);
54         `uvm_info("report_phase", $sformatf("total successful transactions: %d", correct_count), UVM_MEDIUM);
55         `uvm_info("report_phase", $sformatf("total Failed transactions: %d", error_count), UVM_MEDIUM);
56     endfunction
57
58     function void ref_model(alsu_seq_item seq_item_chk);
59
60         //Invalid handling
61
```

```

66
67 //ALU output processing
68 if(seq_item_chk.rst) begin
69     out_ref = 0;
70 end
71 else begin
72     if (seq_item_chk.bypass_A && seq_item_chk.bypass_B)
73         out_ref = (INPUT_PRIORITY == "A")? seq_item_chk.A: seq_item_chk.B;
74     else if (seq_item_chk.bypass_A)
75         out_ref = seq_item_chk.A;
76     else if (seq_item_chk.bypass_B)
77         out_ref = seq_item_chk.B;
78     else if ((seq_item_chk.opcode[1] & seq_item_chk.opcode[2]) | ((seq_item_chk.red_op_A | seq_item_chk.red_op_B) & (seq_item_chk.opcode[1] | seq_item_chk.opcode[2])))
79         out_ref = 0;
80     else begin
81         case (seq_item_chk.opcode) //Fixed Bug: changed incorrect checking opcode to opcode
82         3'h0: begin
83             if (seq_item_chk.red_op_A && seq_item_chk.red_op_B)
84                 out_ref = (INPUT_PRIORITY == "A")? |seq_item_chk.A: |seq_item_chk.B;
85             else if (seq_item_chk.red_op_A)
86                 out_ref = |seq_item_chk.A;
87             else if (seq_item_chk.red_op_B)
88                 out_ref = |seq_item_chk.B;
89             else
90                 out_ref = seq_item_chk.A | seq_item_chk.B;
91         end
92         3'h1: begin
93             if (seq_item_chk.red_op_A && seq_item_chk.red_op_B)
94                 out_ref = (INPUT_PRIORITY == "A")? ^seq_item_chk.A: ^seq_item_chk.B;
95             else if (seq_item_chk.red_op_A)
96                 out_ref = ^seq_item_chk.A;
97             else if (seq_item_chk.red_op_B)
98                 out_ref = ^seq_item_chk.B;
99             else
100                 out_ref = seq_item_chk.A ^ seq_item_chk.B;
101         end
102         3'h2: out_ref = (FULL_ADDER == "ON")? seq_item_chk.A + seq_item_chk.B + seq_item_chk.cin : seq_item_chk.A + seq_item_chk.B ; //Fixed Bug: added FULL_ADDER checker
103         3'h3: out_ref = seq_item_chk.A * seq_item_chk.B;
104         3'h4: begin
105             if (seq_item_chk.direction)
106                 out_ref = {out_ref[4:0], seq_item_chk.serial_in};
107             else
108                 out_ref = {seq_item_chk.serial_in, out_ref[5:1]};
109         end
110         3'h5: begin
111             if (seq_item_chk.direction)
112                 out_ref = {out_ref[4:0], out_ref[5]};
113             else
114                 out_ref = {out_ref[0], out_ref[5:1]};
115         end
116         default: out_ref = 0;
117     endcase
118 end
119 end
120
121 endfunction
122 endclass //alsu_scoreboard extends uvm_scoreboard
123
124 endpackage

```


11. Coverage Collector

```
1  package alsu_CovCollector_pkg;
2
3      import uvm_pkg::*;
4      `include "uvm_macros.svh"
5
6      import alsu_seq_item_pkg::*;
7
8      class alsu_covcollector extends uvm_component;
9          `uvm_component_utils(alsu_covcollector)
10
11          uvm_analysis_export #(alsu_seq_item) cov_export;
12          uvm_tlm_analysis_fifo #(alsu_seq_item) cov_fifo;
13          alsu_seq_item seq_item;
14
15
16          covergroup CovCode;
17
18              A_CP: coverpoint seq_item.A {
19                  bins A_data_0 = {0};
20                  bins A_data_max = {MAXPOS};
21                  bins A_data_min = {MAXNEG};
22                  bins A_data_default = default;
23                  bins A_data_walkingones[] = {3'b001, 3'b010, 3'b100} iff(seq_item.red_op_A);
24              }
25
26              B_CP: coverpoint seq_item.B {
27                  bins B_data_0 = {0};
28                  bins B_data_max = {MAXPOS};
29                  bins B_data_min = {MAXNEG};
30                  bins B_data_default = default;
31                  bins B_data_walkingones[] = {3'b001, 3'b010, 3'b100} iff(seq_item.red_op_B && ~seq_item.red_op_A);
32              }
33
34
35              ALU_cp: coverpoint seq_item.opcode {
36                  bins Bins_shift[] = {SHIFT, ROTATE};
37                  bins Bins_arith[] = {ADD, MULT};
38                  bins Bins_bitwise[] = {OR, XOR};
39                  bins Bins_trans = {0 => 1 => 2 => 3 => 4 => 5};
40                  illegal_bins Bins_invalid = {INVALID_6, INVALID_7};
41              }
42
43              //Added coverpoint for C_in
44              cin_CP: coverpoint seq_item.cin
45              { option.weight = 0; }
46
47              //Added coverpoint for direction
48              direction_CP: coverpoint seq_item.direction{
49                  option.weight = 0;
50              }
51
52              //Added coverpoint for serial_in
53              serial_in_CP: coverpoint seq_item.serial_in{
54                  option.weight = 0;
55              }
56
57              //Added coverpoint for red_op_A
58              red_op_A_cp: coverpoint seq_item.red_op_A{
59                  option.weight = 0;
60              }
61
62              //Added coverpoint for red_op_B
63              red_op_B_cp: coverpoint seq_item.red_op_B{
64                  option.weight = 0;
65              }
66
67
68
69              //1.Cross coverage for ADD and MULT with A/B permutations
70              A_B_ALU_cross: cross A_CP, B_CP, ALU_cp {
71
72                  bins add_mult_perms = binsof(ALU_cp.Bins_arith) &&
73                      binsof(A_CP) intersect {MAXPOS, MAXNEG, 0} &&
74                      binsof(B_CP) intersect {MAXPOS, MAXNEG, 0};
75                  option.cross_auto_bin_max = 0;
76              }
77
```

```

78      //2.ALU is addition, c_in should have taken 0 or 1
79      CIN_ADD_cross: cross ALU_cp, cin_CP{
80
81          bins cin_add = binsof(ALU_cp.Bins_arith) intersect {ADD} &&
82                      binsof(cin_CP) intersect {0, 1};
83          option.cross_auto_bin_max = 0;
84      }
85
86
87      //3.ALSU is shifting or rotating, then direction must take 0 or 1
88      DIRECTION_ALU_cross: cross direction_CP, ALU_cp {
89
90          bins direction_ALU = binsof(ALU_cp.Bins_shift) &&
91                          binsof(direction_CP) intersect {0, 1};
92          option.cross_auto_bin_max = 0;
93      }
94
95      //4.ALSU is shifting, then shift_in must take 0 or 1
96      SERIAL_IN_SHIFT_cross: cross serial_in_CP, ALU_cp {
97          bins serial_in_ALU = binsof(ALU_cp.Bins_shift) intersect {SHIFT} &&
98                          binsof(serial_in_CP) intersect {0, 1};
99          option.cross_auto_bin_max = 0;
100     }
101
102     //5.ALSU is OR or XOR and red_op_A is asserted, then A took all walking one patterns
103     //(001, 010, and 100) while B is taking the value 0
104     RED_OP_A_OR_XOR_cross: cross ALU_cp, A_CP, B_CP {
105         bins red_op_A_or_xor = binsof(ALU_cp.Bins_bitwise) &&
106                             binsof(A_CP.A_data_walkingones) &&
107                             binsof(B_CP.B_data_0);
108         option.cross_auto_bin_max = 0;
109     }
110
111     //6.ALSU is OR or XOR and red_op_B is asserted, then B took all walking one patterns
112     //(001, 010, and 100) while A is taking the value 0
113     RED_OP_B_OR_XOR_cross: cross ALU_cp, A_CP, B_CP {
114         bins red_op_B_or_xor = binsof(ALU_cp.Bins_bitwise) &&
115                             binsof(B_CP.B_data_walkingones) &&
116                             binsof(A_CP.A_data_0);
117         option.cross_auto_bin_max = 0;
118     }
119
120     //7.invalid case: reduction operation is activated while the opcode is not OR or XOR
121     RED_INVLAID_cross: cross ALU_cp, red_op_A_cp, red_op_B_cp {
122         ignore_bins OR_XOR = binsof(red_op_A_cp) &&
123                             binsof(red_op_B_cp) &&
124                             binsof(ALU_cp.Bins_bitwise);
125
126         ignore_bins trans = binsof(ALU_cp.Bins_trans);
127     }
128
129     endgroup
130
131     function new(string name = "alsu_covcollector", uvm_component parent = null);
132         super.new(name, parent);
133         CovCode = new;
134     endfunction //new()
135
136     function void build_phase(uvm_phase phase);
137         super.build_phase(phase);
138         cov_export = new("cov_export", this);
139         cov_fifo = new("cov_fifo", this);
140     endfunction
141
142     function void connect_phase(uvm_phase phase);
143         super.connect_phase(phase);
144         cov_export.connect(cov_fifo.analysis_export);
145     endfunction
146
147     task run_phase(uvm_phase phase);
148         super.run_phase(phase);
149         forever begin
150             cov_fifo.get(seq_item);
151             CovCode.sample();
152         end
153     endtask
154
155     endclass //alsu_CovCollector extends uvm_component
156
157     endpackage

```

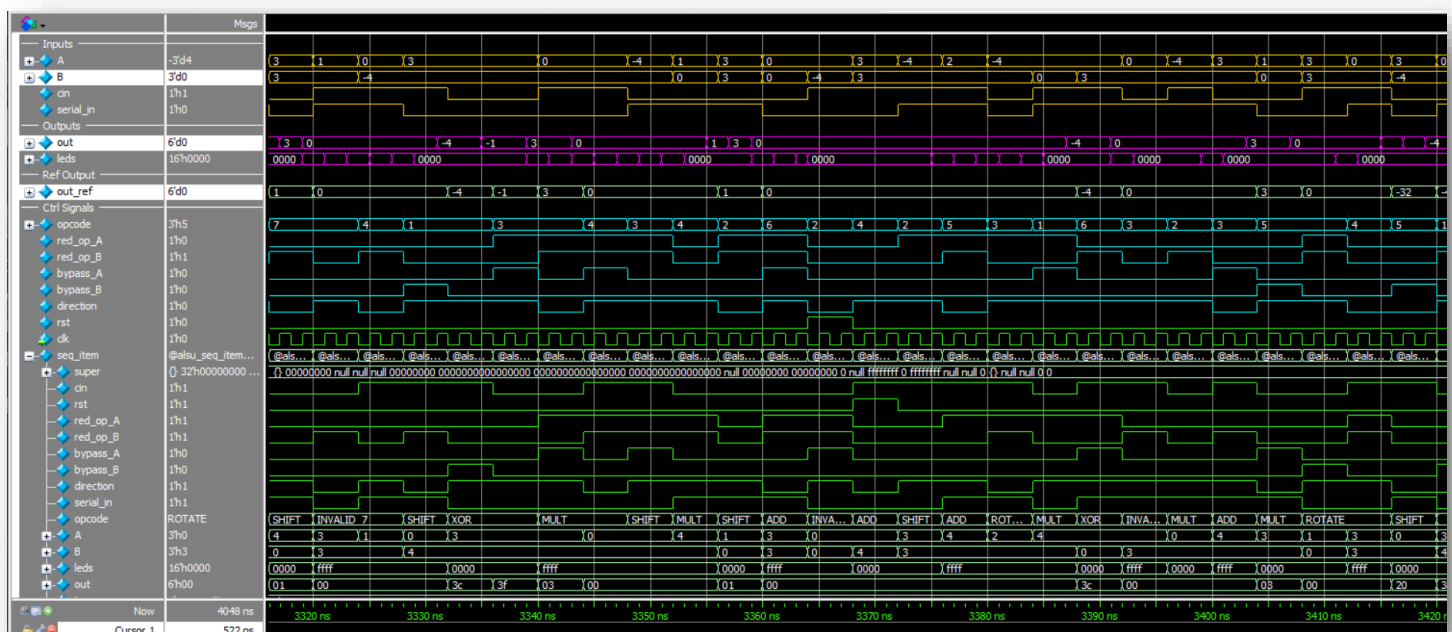
Questasim Output

1. Transcript Output

```
# UVM_INFO uvm/alsu_test.sv(51) @ 4048: uvm_test_top [run_phase] Driected Sequencer end
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 4048: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO uvm/alsu_scoreboard.sv(54) @ 4048: uvm_test_top.env.sb [report_phase] total successful transactions: 1009
# UVM_INFO uvm/alsu_scoreboard.sv(55) @ 4048: uvm_test_top.env.sb [report_phase] total Failed transactions: 3
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 12
# UVM_WARNING : 0
# UVM_ERROR : 3
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [report_phase] 2
# [run_phase] 9
# ** Note: $finish : D:/Mina/Program_Files/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 4048 ns Iteration: 61 Instance: /top
# 1
# Break in Task uvm_pkg/uvm_root::run_test at D:/Mina/Program_Files/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
VSTM(paused)\>
```

Note: the 3 error is due to SHIFT and ROTATE operation done twice in 2 clock cycles, which is expected form the RTL design but I couldn't modify the scoreboard to handle it.

2. Waveform



3. Assertions

▲ /uvm_pkg::uvm_reg_map::do_write/#ublk#215181159#1731/immed__1735	Immediate	SVA	on	0	0	-	-	-
▲ /uvm_pkg::uvm_reg_map::do_read/#ublk#215181159#1771/immed__1775	Immediate	SVA	on	0	0	-	-	-
▲ /aslu_seq_pkg::alsu_main_seq::body/#ublk#256728695#43/immed__46	Immediate	SVA	on	0	1	-	-	-
▲ /aslu_seq_pkg::alsu_directed_seq::body/#anonblk#256728695#65#4#/#ublk#256728695#6...	Immediate	SVA	on	0	1	-	-	-
+ ▲ /top/DUT/ALSU_SVA_inst/p_invalid_out_zero_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/p_valid_leds_zero_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/bypassA_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/bypassB_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/red_op_A_xor_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/red_op_A_or_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/red_op_B_xor_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/red_op_B_or_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/OR_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/XOR_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/ADD_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/MUL_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/SHIFT_R_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/ROTATE_R_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/SHIFT_L_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/ROTATE_L_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/invalid_assertion	Concurrent	SVA	on	0	1	-	0B	0B
+ ▲ /top/DUT/ALSU_SVA_inst/#ublk#25906369#19/immed__20	Immediate	SVA	on	0	1	-	-	-

4. Assertion Coverage

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory
▲ /top/DUT/ALSU_SVA_inst/#ublk#25906369#19/i_cover__21	SVA	✓	Off	40	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/p_invalid_out_zero_coverage	SVA	✓	Off	960	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/p_valid_leds_zero_coverage	SVA	✓	Off	679	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/bypassA_coverage	SVA	✓	Off	190	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/bypassB_coverage	SVA	✓	Off	190	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/red_op_A_xor_coverage	SVA	✓	Off	16	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/red_op_A_or_coverage	SVA	✓	Off	2	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/red_op_B_xor_coverage	SVA	✓	Off	38	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/red_op_B_or_coverage	SVA	✓	Off	2	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/OR_coverage	SVA	✓	Off	2	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/XOR_coverage	SVA	✓	Off	182	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/ADD_coverage	SVA	✓	Off	64	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/MUL_coverage	SVA	✓	Off	74	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/SHIFT_R_coverage	SVA	✓	Off	34	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/ROTATE_R_coverage	SVA	✓	Off	28	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/SHIFT_L_coverage	SVA	✓	Off	38	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/ROTATE_L_coverage	SVA	✓	Off	38	1	Unli...	1	100%	██████████	✓	0
▲ /top/DUT/ALSU_SVA_inst/invalid_coverage	SVA	✓	Off	1192	1	Unli...	1	100%	██████████	✓	0

5. Functional Coverage

INST Valsu_CovCollector_pkg::alsu_cov...	100.00%	100	100.00...	██████████	✓
+ CVP A_CP	100.00%	100	100.00...	██████████	✓
+ CVP B_CP	100.00%	100	100.00...	██████████	✓
+ CVP ALU_cp	100.00%	100	100.00...	██████████	✓
+ CVP cin_CP	100.00%	100	100.00...	██████████	✓
+ CVP direction_CP	100.00%	100	100.00...	██████████	✓
+ CVP serial_in_CP	100.00%	100	100.00...	██████████	✓
+ CVP red_op_A_cp	100.00%	100	100.00...	██████████	✓
+ CVP red_op_B_cp	100.00%	100	100.00...	██████████	✓
+ CROSS A_B_ALU_cross	100.00%	100	100.00...	██████████	✓
+ CROSS CIN_ADD_cross	100.00%	100	100.00...	██████████	✓
+ CROSS DIRECTION_ALU_cross	100.00%	100	100.00...	██████████	✓
+ CROSS SERIAL_IN_SHIFT_cross	100.00%	100	100.00...	██████████	✓
+ CROSS RED_OP_A_OR_XOR_cross	100.00%	100	100.00...	██████████	✓
+ CROSS RED_OP_B_OR_XOR_cross	100.00%	100	100.00...	██████████	✓
+ CROSS RED_INVLAID_cross	100.00%	100	100.00...	██████████	✓